Chapter 6

樹狀結構

6.1 樹狀結構的基本概念

- 「樹狀結構」(Tree Structure) 的優點在於層次分明並且有條理，可以清楚地表示出資料之間上、下、先、後的從屬關係。
- 樹 (Tree) 由一個特殊的節點或許多個節點所組成的集合。一棵樹包含一個樹根節點 (Root) ，以及 N 個互斥的子樹 (Subtree)， N≧0 。
- 樹的節點分為樹葉節點 (Leaf) 又稱為終端節點 (Terminal Nodes) 以及非終端節點 (Nonterminal Nodes) 兩種。

  沒有再分出任何子樹的節點稱為樹葉節點，反之則稱為非終端節點。

6.1 樹狀結構的基本概念

【範例】一棵分支度為 3 的樹，樹根為 A ，樹葉節點有 E、F、I、J、K、M 等 6 個節點，至於 A、B、C、D、G、H、L 為非終端節點。

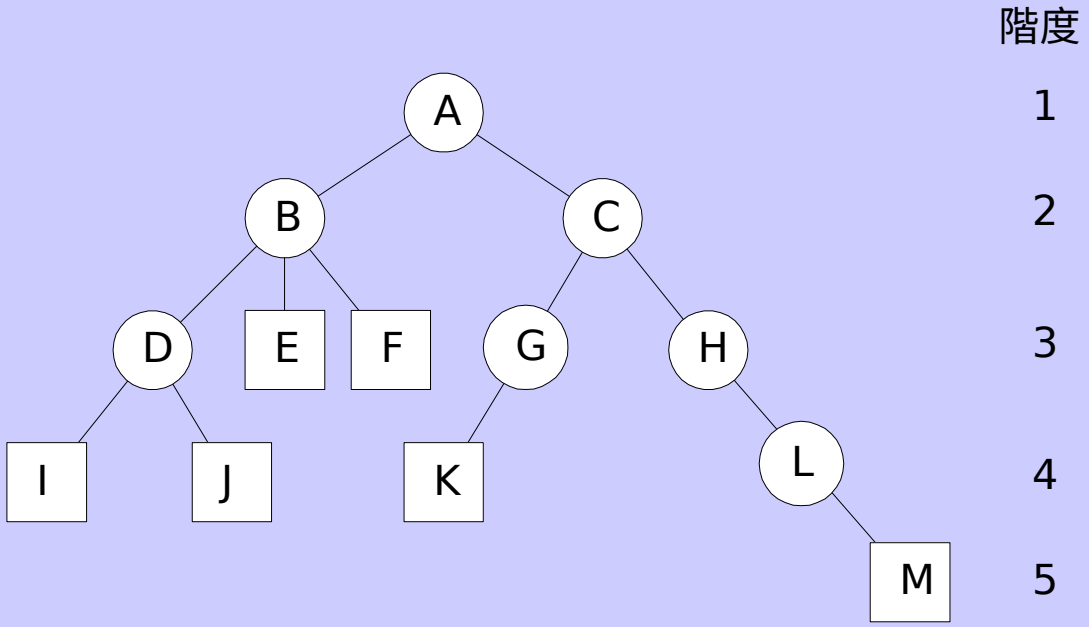


圖 6.1 樹的範例

6.2 樹狀結構的表示法

- 表示一個給定的樹狀結構最簡單的方法就是利用繪圖的方式來記錄此樹狀結構。

- 樹狀結構利用電腦來表示與處理常見的方法有兩類：串列的表示法與左子右弟表示法。

6.2.1 串列的表示法

- 樹狀結構可用樹根節點後接著一串樹根節點的子樹群來表示。個別子樹又可利用子樹的樹根節點跟著一連串的次子樹節點來表示。例如圖 6.1 的樹狀結構可表示為：

A(B(D(I,J),E),F,C(G(K),H,L,M))

- 我們可以利用圖 6.2 所描述的鏈結串列之節點結構來表示樹狀結構的資料。

節點資料	鏈結至 子樹1	鏈結至 子樹2	鏈結至 子樹3	. . .	鏈結至 子樹d

圖 6.2　用來表示樹的鏈結串列結構

6.2.1 串列的表示法

- 圖 6.3 的表示法是將圖 6.1 的樹狀結構利用圖 6.2 的鏈結串列結構延伸出來的。

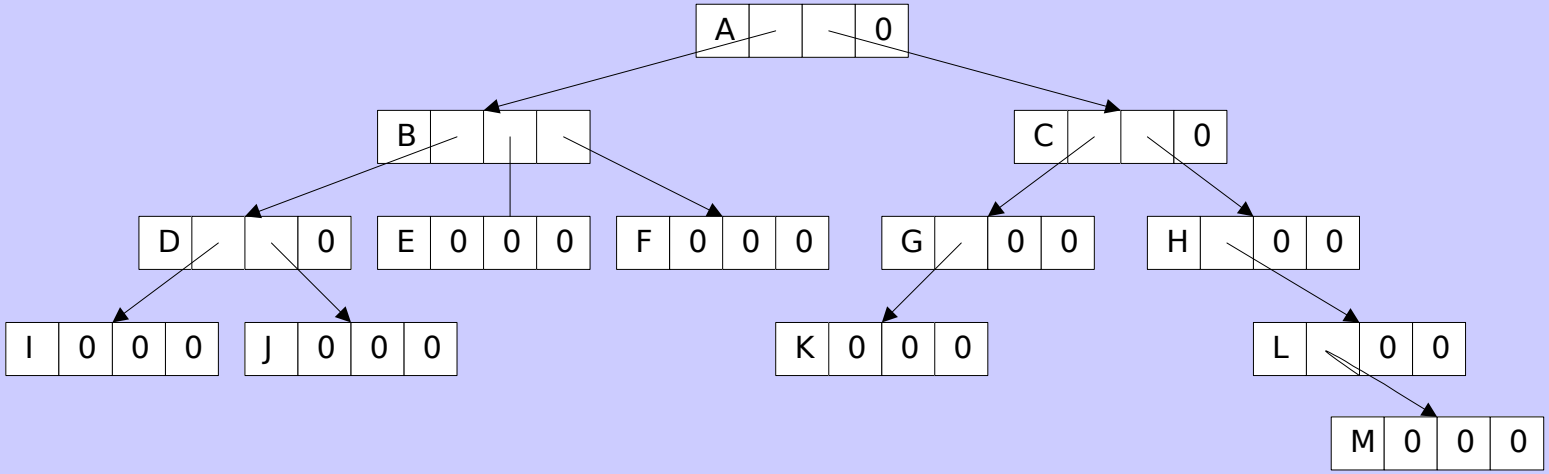


圖 6.3　圖 6.1 的鏈結表示法

- 缺點：圖 6.3 的鏈結串列結構較浪費記憶體，因為有很多鏈結根本用不上（如圖 6.3 中節點之鏈結欄為 0 的部份）。

6.2.2 左子 - 右弟表示法

- 所謂「左子 - 右弟表示法」乃是將一般樹轉換成二元樹來表示。

- 表示二元樹的每一節點之鏈結串列結構如圖 6.4 所示 。

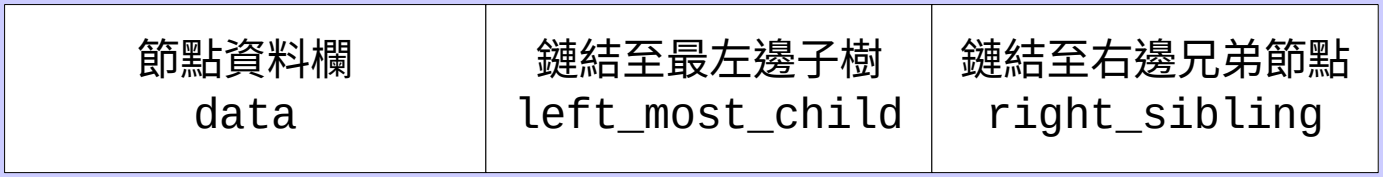


圖 6.4　二元樹中每一節點的鏈結串列結構

6.2.2 左子 - 右弟表示法

- 將給定的 d 元樹轉換成二元樹的步驟如下所列：

步驟 (1)：每一個節點除了鏈結至最左子樹之鏈結外，餘均予刪除。

步驟 (2)：每一節點均鏈結至其右邊之兄弟節點。

【範例】圖 6.5(a) 是一棵分支度為 3 的樹，其轉換成二元樹的過程如圖 6.5(b) 及圖 6.5(c) 所示，而圖 6.24(d) 則是將圖 6.5(c) 順時針旋轉 45 度後的結果。

6.2.2 左子 - 右弟表示法

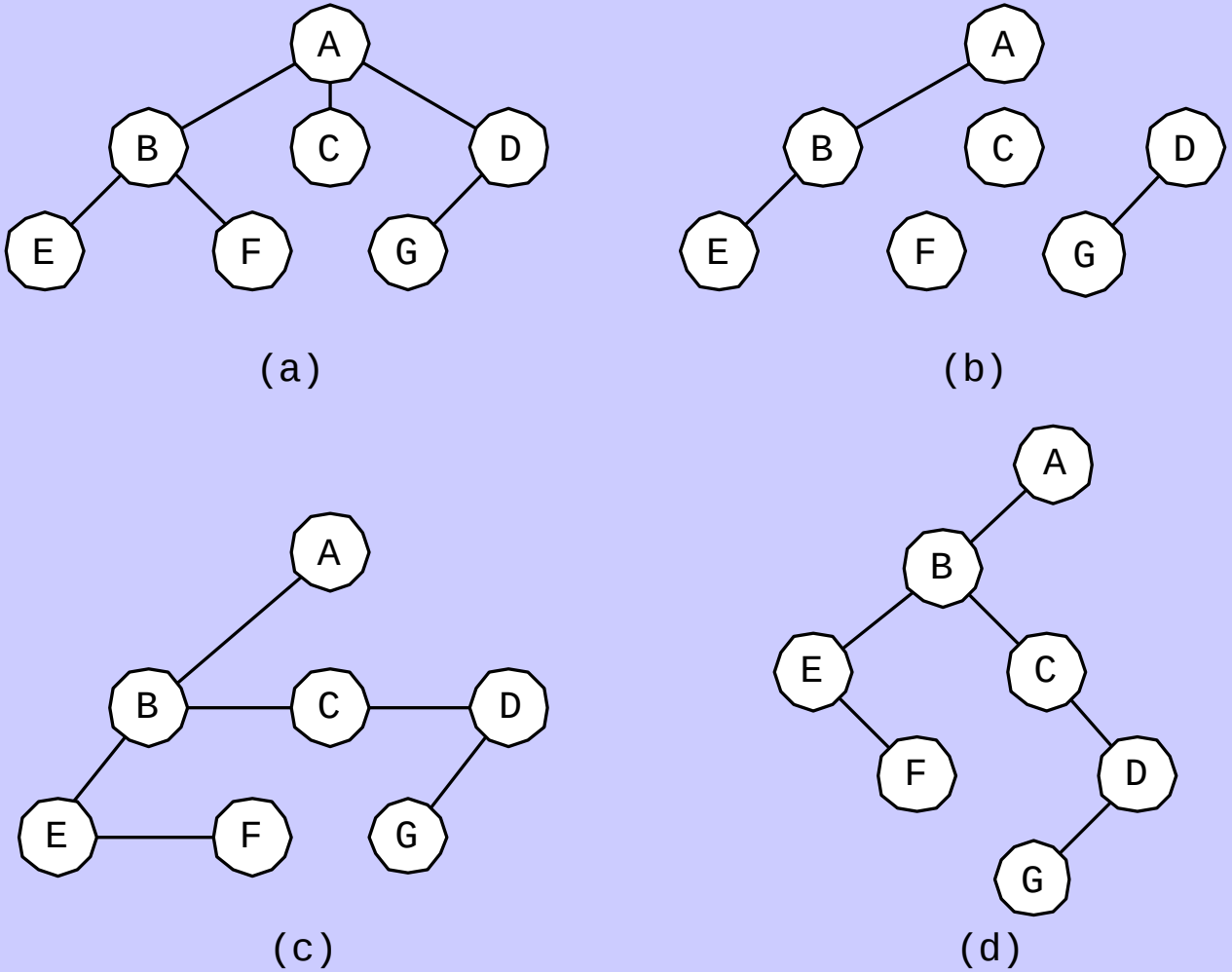


圖 6.24 一棵樹轉換成一棵二元樹的過程

6.3 二元樹 (Binary Tree)

- 二元樹可以是一棵空樹，或是由一個樹根以及包含兩個互相分離的樹（稱為左子樹和右子樹）所組成。

- 個別子樹可以是一棵空樹，而且子樹有左右之分，所以二元樹又被稱為有序樹 (Ordered Tree)。

- 二元樹可以是空集合；而非空集合的樹至少要有一個樹根。

- 基本上一棵二元樹是一個樹狀結構其樹的分支度小於等於 2。

6.3 二元樹 (Binary Tree)

- 圖 6.6 展示了幾個可能的二元樹類型 ：

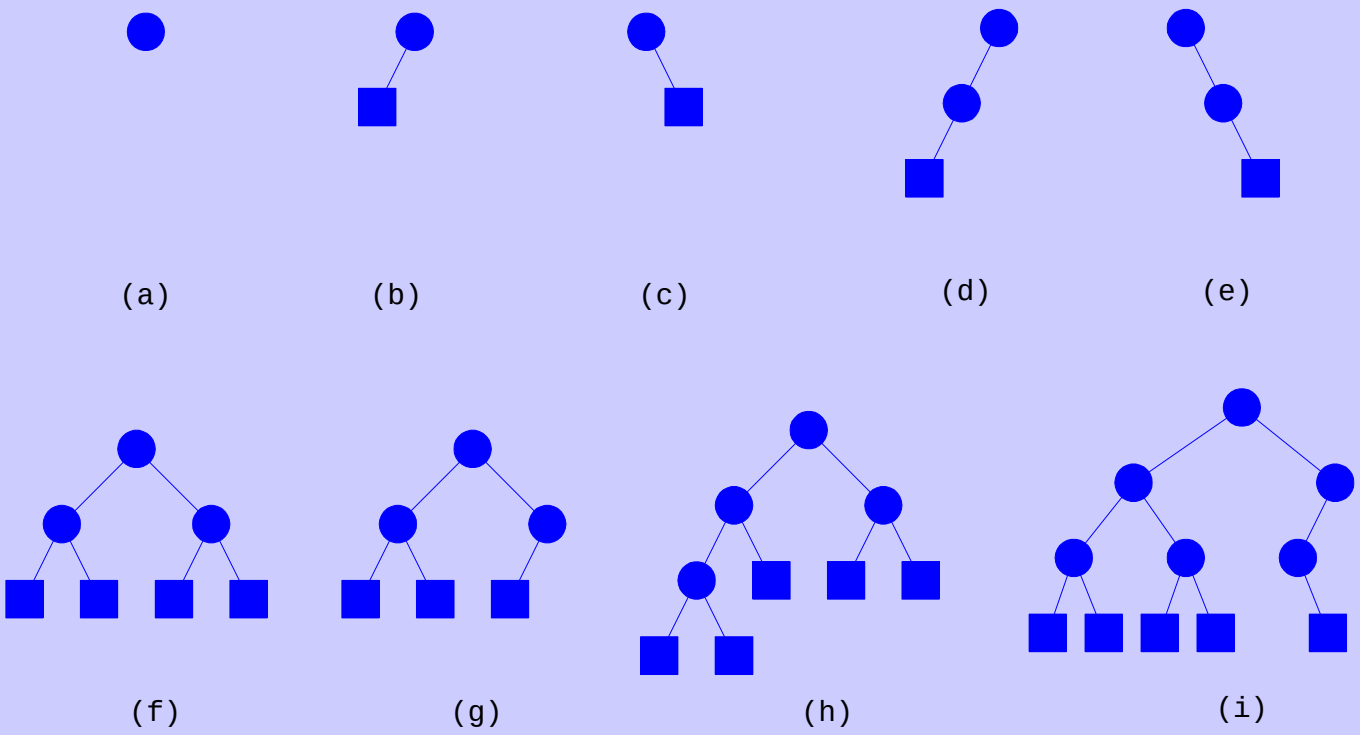


圖 6.6　二元樹

6.3 二元樹 (Binary Tree)

- 一棵二元樹的所有非終端節點的分支度皆為二，我們將它稱為完滿二元樹 (Full Binary Tree) ，如圖 6.6(f) 便叫做完滿二元樹。

- 完整 (Complete Binary Tree) 二元樹的樹葉節點不是落在第 k 階層就是落在 (k-1) 階層，且樹中在 (k-1) 階層的任一非終端節點若存在右子節點於第 k 階層則其左子節點一定存在第 k 階層。

- 完整二元樹的節點總數會小於等於相同深度的完滿二元樹的節點總數。

6.3 二元樹 (Binary Tree)

- 二元樹的一般特性可以歸納如下：

1. 第 i 階層的總節點數小於等於 2^{i-1} ， i≧1 。
2. 二元樹的節點總數少於 2^{K} ， k 為二元樹的深度， k≧1 。
3. 樹葉節點總數等於分支度為 2 的節點總數加 1 。

- 一棵深度為 k 的二元樹其節點總數最多為 $2^{K}-1$ ，也就是這棵樹是一個完滿二元樹。

6.4 二元樹的表示法

- 在樹狀結構的表示法中因為二元樹分支度小於等於二，使得二元樹的表示較一般的樹狀結構來得容易。

- 常見的兩種二元樹的表示法為鏈結串列表示法和陣列表示法。

6.4.1 鏈結串列表示法

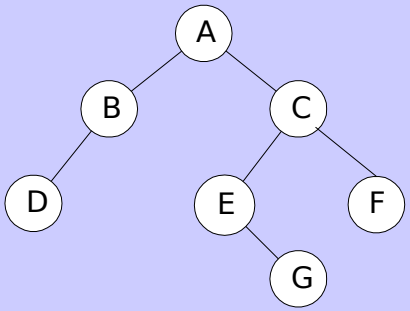
- 利用鏈結串列來表示一棵二元樹，我們可以利用圖 6.7 所描述的節點結構來記錄二元樹個別節點的資料。

左子樹鏈結 l_child	節點資料欄 data	右子樹鏈結 r_child

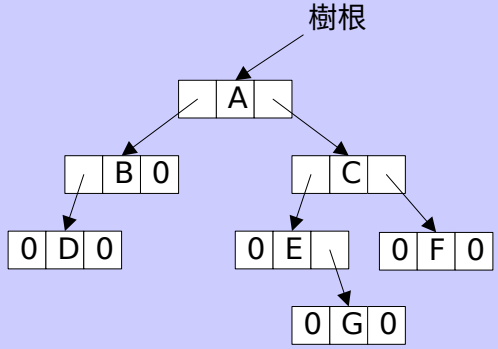
圖 6.7　二元樹的節點結構

【範例】圖 6.8(a) 之二元樹可以利用圖 6.7 的節點結構轉換成圖 6.8(b) 的結果。當您要把 6.8(a)G 節點的父節點由 E 改為 F 時，如圖 6.8(c) ，您只需要調整鏈結指標即可。

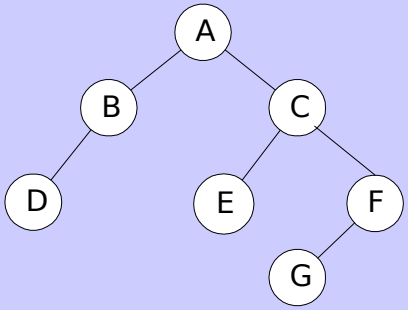
6.4.1 鏈結串列表示法



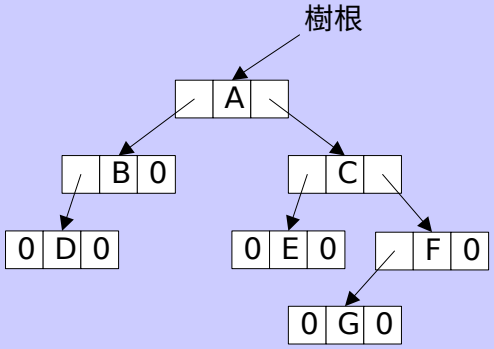
(a) 二元樹



(b) 二元樹的鏈結表示法



(c)

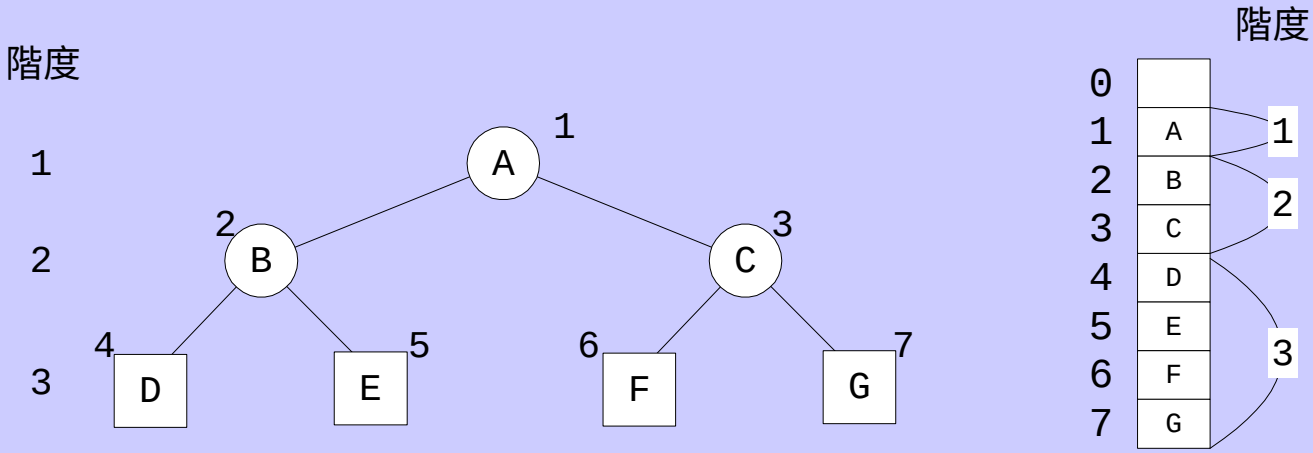


(d)

圖 6.8 二元樹的鏈結表示法

6.4.2 一維陣列表示法

● 圖 6.9(b) 描述了圖 6.9(a) 之完滿二元樹的陣列表示法。



(a) 完滿二元樹　　(b) 完滿二元樹的陣列表示法

圖 6.9 完滿二元樹的陣列表示法

6.4.2 一維陣列表示法

【範例】二元樹的陣列表示法。

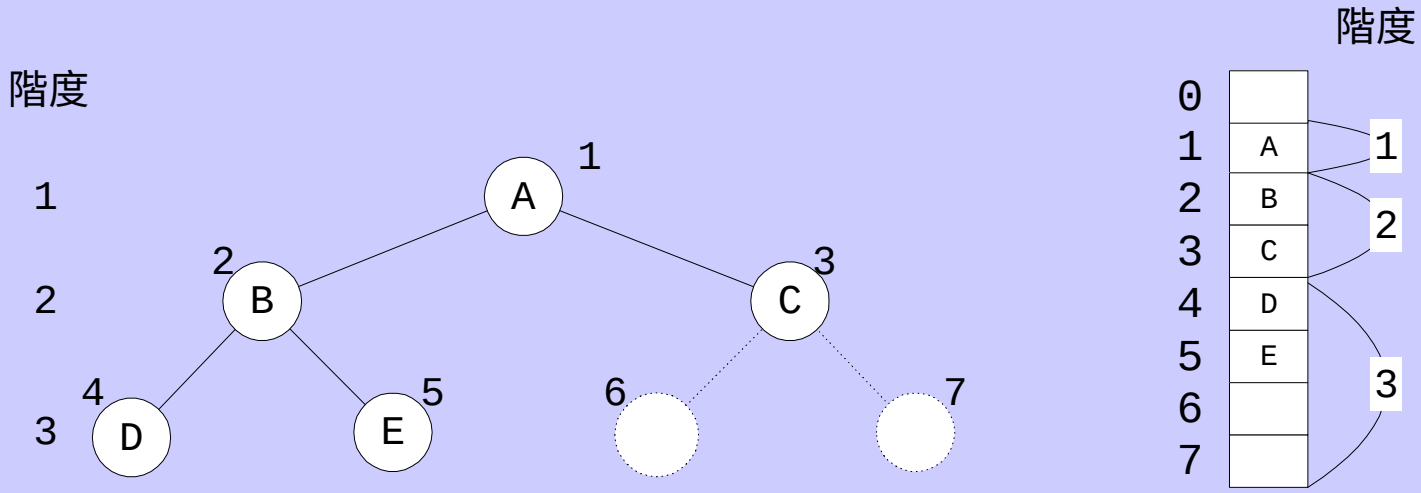


圖 6.10　二元樹的陣列表示法

6.4.2 一維陣列表示法

- 陣列表示法可能很節省空間（當二元樹接近完滿時），也可能非常浪費空間。當二元樹很接近完滿二元樹時，陣列表示法就非常省空間；然而，當二元樹是如歪斜樹時，陣列表示法就相當浪費空間，原因在於陣列中會出現許多空值。

- 陣列表示法可以很容易算出某一節點在陣列中之位置，並且可以很快速地找出相關父子節點之位置。

- 陣列表示法在處理節點的插入和刪除時必須搬動大量資料，這是它的一個大缺點。

6.4.3 二維陣列表示法

- 除了可以利用一維陣列表示二元樹之外，我們也可以利用二維陣列來表示給定的二元樹。

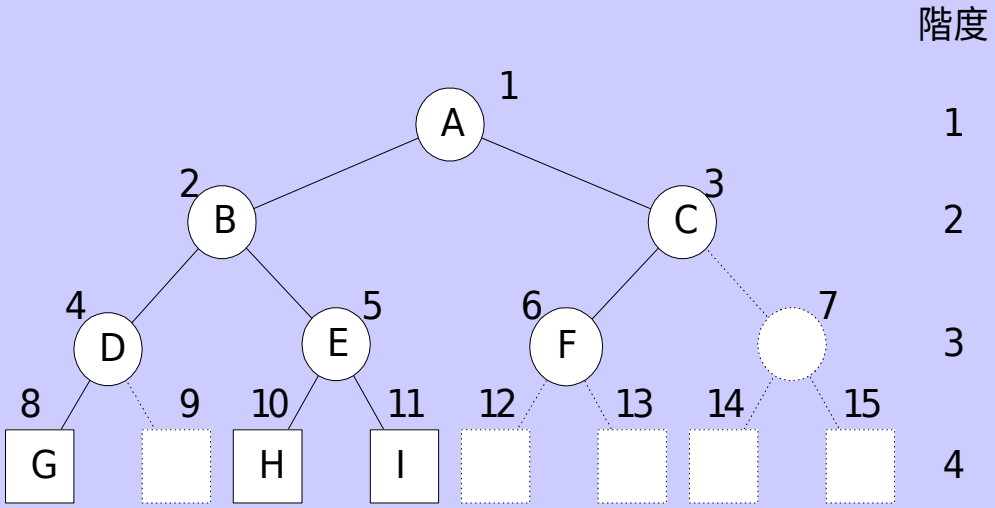


圖 6.11 二元樹範例

6.4.3 二維陣列表示法

表 6.1　圖 6.11 二元樹之二維陣列表示法

	左子樹位置	節點資料	右子樹位置
1	2	A	3
2	4	B	5
3	6	C	0
4	8	D	0
5	10	E	11
6	0	F	0
7	0	G	0
8	0	H	0
9	0	I	0

6.5 二元樹的追蹤和應用

- 給定一棵二元樹，如果我們希望拜訪所有的節點並將個別節點的資料輸出稱為二元樹的「追蹤」(Traversal)。

- 如果我們事先規定以「先左子樹後右子樹」的順序來進行追蹤，那麼二元樹的追蹤可再細分為 LDR 、 LRD 以及 DLR 三種次序，其中 L 代表左子樹， D 代表節點資料，R 代表右子樹。

6.5 二元樹的追蹤和應用

【範例】二元樹的追蹤

給定一棵如圖 6.12 所示包含 10 個節點的二元樹。其中序追蹤結果為：B、A、F、D、H、G、J、I、C、E。前序追蹤結果為：A、B、C、D、F、G、H、I、J、E。後序追蹤結果為：B、F、H、J、I、G、D、E、C、A。

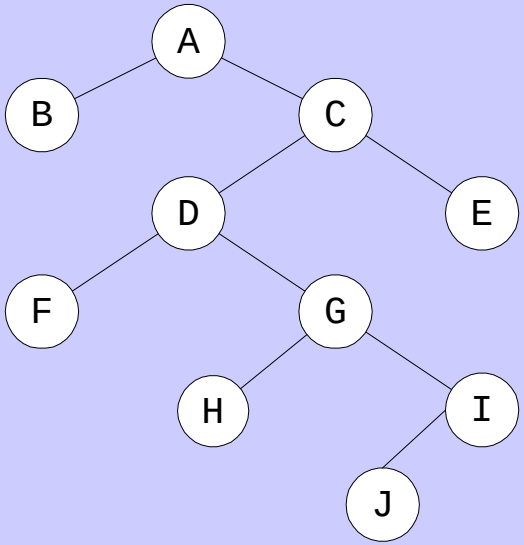


圖 6.12

6.5 二元樹的追蹤和應用

【範例】將完整括號的中序式運算式 ((A-B)+(C*(D-E)/F)) 轉換成對應二元樹的處理過程。

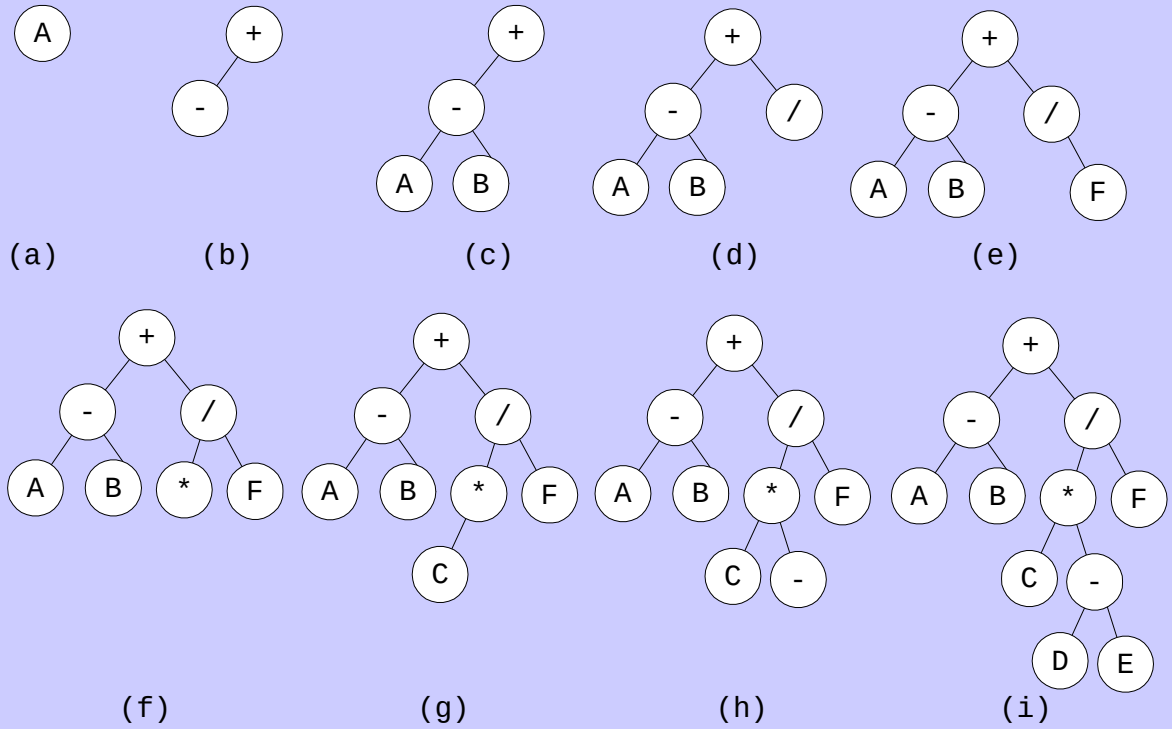


圖 6.18 將完整括號的中序式運算式 ((A-B)+(C*(D-E)/F)) 轉換成對應二元樹的處理過程

6.6 引線二元樹 (Tread Binary Tree)

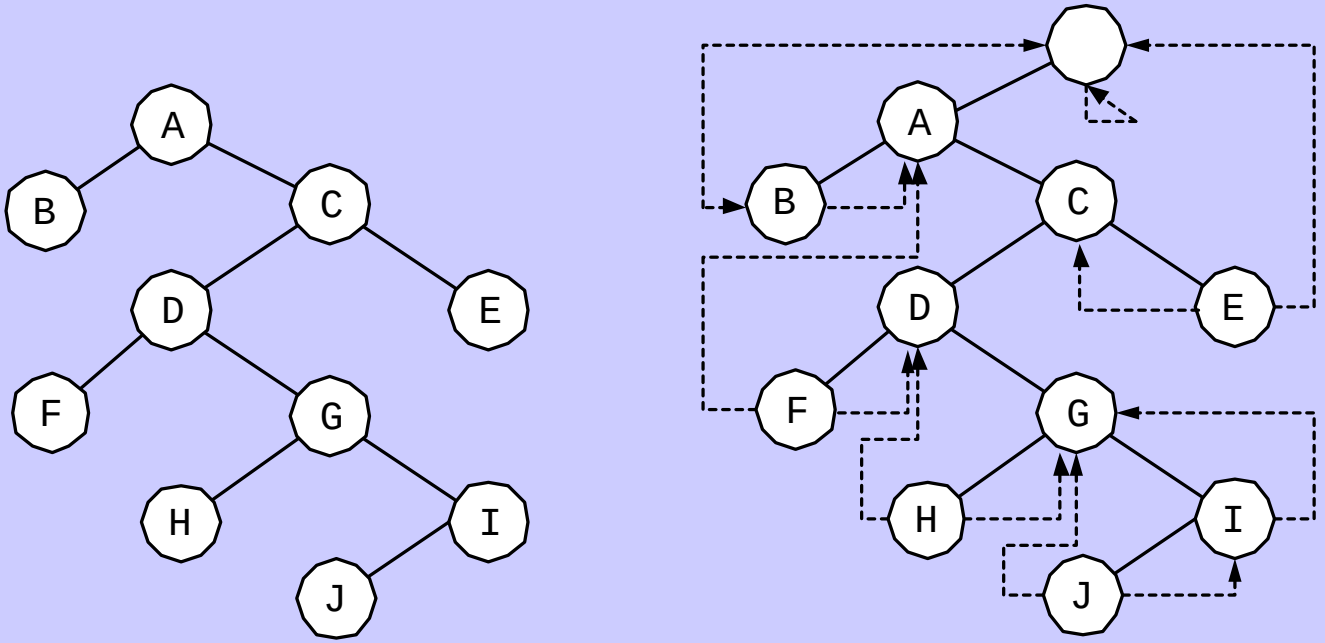
- 所謂的引線 (Threads) 指的就是將原本樹葉節點以及非終端節點未使用到的鏈結欄位設定到其他節點的記憶體位置。
- 給定的一棵二元樹轉換為對應的引線二元樹的處理程序列於下：

步驟 (1)：虛設一個空節點。並假設引線二元樹是這個虛設的空節點的左子樹，這虛設節點的右子樹鏈結指向本身。

步驟 (2)：將樹葉節點以及非終端節點未使用到的左子樹鏈結指向中序追蹤的前一個節點。

步驟 (3)：將樹葉節點以及非終端節點未使用到的右子樹鏈結指向中序追蹤的下一個節點。

6.6 引線二元樹 (Tread Binary Tree)



(a) 二元樹　　(b) 引線二元樹

圖 6.19　引線二元樹轉換範例

6.6 引線二元樹 (Tread Binary Tree)

- 為了要區分使用鏈結串列結構來表示引線二元樹時，個別節點的左（右）子樹鏈結欄到底是一般指到左（右）子樹的鏈結，或者是被當成引線的鏈結來使用。我們將圖 6.7 的鏈結串列節點結構稍加修正如圖 6.20 所描述的節點結構。

左控制位元 l_flag	左子樹鏈結 l_child	節點資料欄 data	右子樹鏈結 r_child	右控制位元 r_flag

圖 6.20　引線二元樹的表示法

6.6 引線二元樹 (Tread Binary Tree)

- 我們重新描繪圖 6.19(b) 為圖 6.21 之完整鏈結結構。

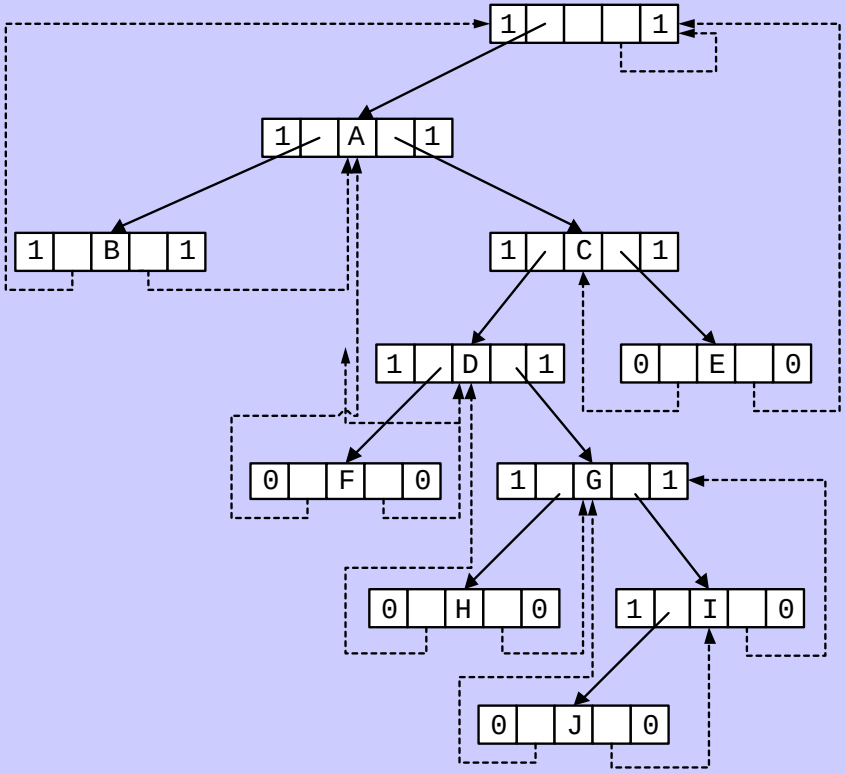


圖 6.21　加入控制欄位之引線二元樹

6.6 引線二元樹 (Tread Binary Tree)

- 引線二元樹的節點插入方法如下：

```
    1. 以任何一種追蹤法找到 X 節點位置。
2. 將 X 指向右子樹的鏈結 ( 或引線 ) 移轉給 Y ，即
      Y->r_child = X->r_child;
      Y->r_flag = X->r_flag 。
3. Y 的左子樹鏈結應是一指向 X 的引線，即
      Y->l_child = X;
      Y->l_flag = 0;
4. Y 成為 X 的右子樹根，即
      X->r_child = Y;
      X->r_flag = 1;
5. 若 X 有右子樹時，必須將 X 的後繼者之左子樹鏈結指向 Y ，即
      if(Y->r_flag == 1)
  {
          Z = Successor(Y);
    Z->l_child = Y;
  }
```

6.6 引線二元樹 (Tread Binary Tree)

- Successor(Y) 函數是用來找出節點 Y 的後繼節點 S ，其處理程序如下所列：

```
1. S = Y->r_child;
2. if(Y->r_flag == 0)
     return S;
3. while(S->l_flag == 1)
     S = S->l_child;
4. return S;
```

【範例】插入一個新節點 X 到圖 6.21 之引線二元樹中節點 C 的右邊成為節點 C 的右子樹根之結果如圖 6.22 ，其中①至④為鏈結更動之步驟 。

6.6 引線二元樹 (Tread Binary Tree)

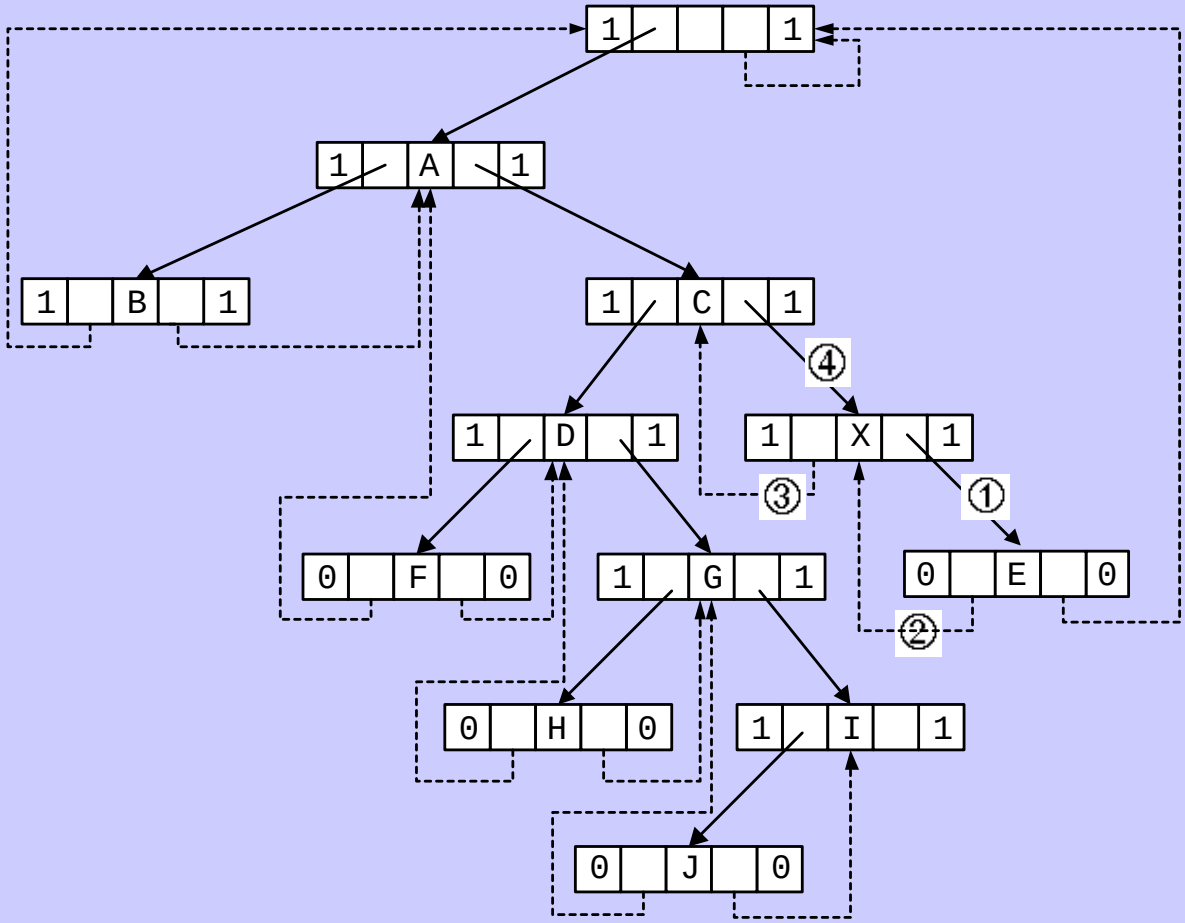


圖 6.22　插入新節點 X 後的引線二元樹

6.7 二元搜尋樹、平衡樹、和 B 樹

- 除了上述各節介紹的二元樹之外，二元搜尋樹、平衡樹以及 B 樹也是另外三種常見的樹狀結構，三者都可以用來協助資料的搜尋，其基本概念將分別在以下各節予以說明。

6.7.1 二元搜尋樹 (Binary Search Tree)

- 二元搜尋樹 T 是一棵二元樹，每一個節點上均存放一個鍵值資料，並且樹根之鍵值大於左子樹的所有鍵值，小於或等於右子樹的所有鍵值，並且左右子樹也是二元搜尋樹。

【範例】給定一組尚未排序的鍵值資料 {31 ， 76 ， 11 ， 47 ， 20 ， 55 ， 33 ， 120} ，建構二元搜尋樹的過程如下

：

6.7.1 二元搜尋樹 (Binary Search Tree)

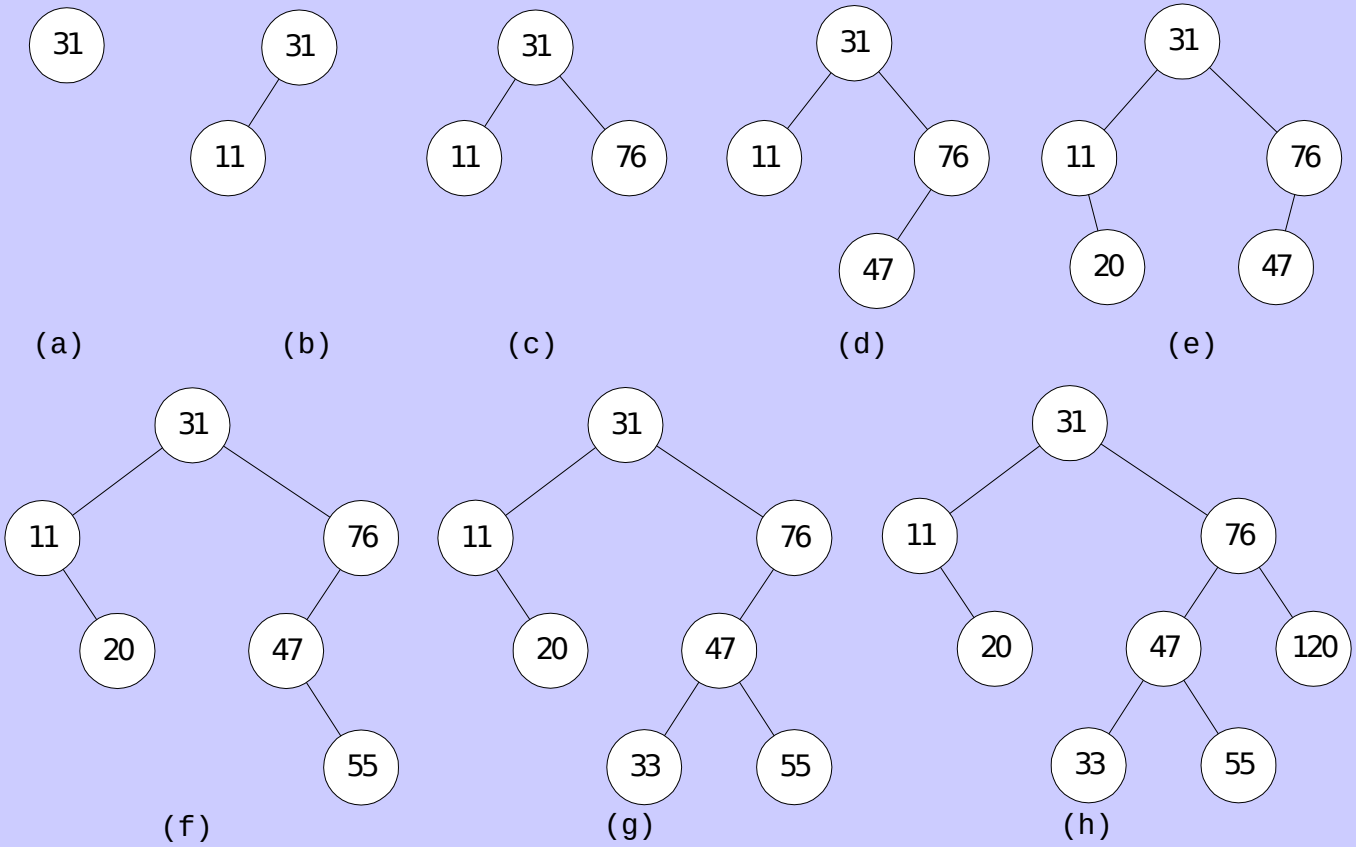


圖 6.23　建立二元搜尋樹的處理過程

6.7.1 二元搜尋樹 (Binary Search Tree)

- 二元搜尋樹刪除一筆資料可以分成兩類來處理：要刪除的資料位於樹葉節點與要刪除的資料位於非終端節點。

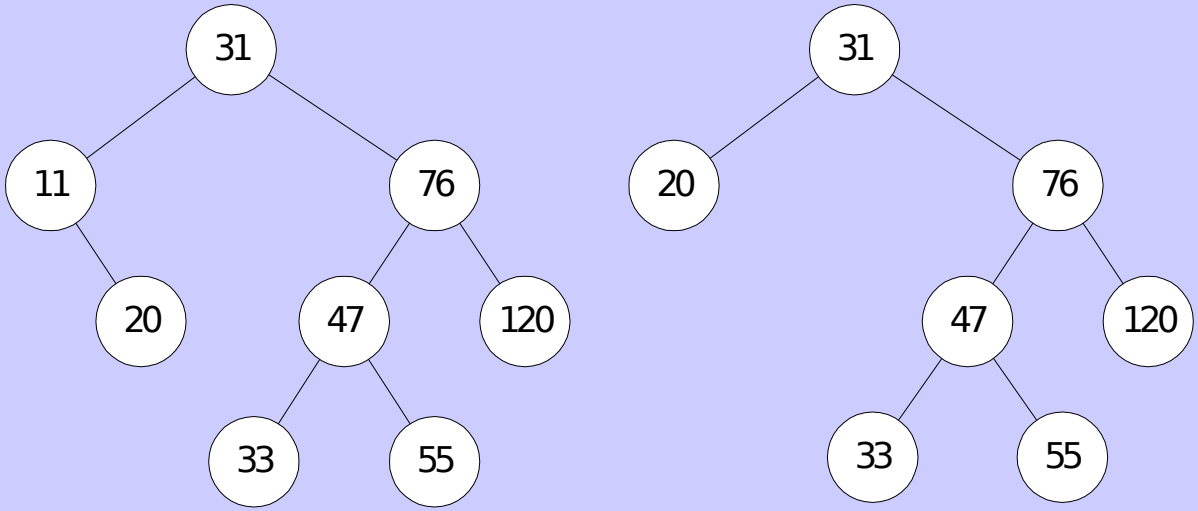
- 若是要刪除的資料位於樹葉節點是較簡單的，可以直接刪除掉這筆資料因為它不會影響到二元搜尋樹的特性。

- 刪除位於非終端節點的資料，因為非終端節點的分支度可能為 1 或 2 ，不同分支度的非終端節點在處理上稍有不同。

1. 舉例來說如果我們希望從圖 6.24(a) 所描述的二元搜尋樹刪除 11 這筆資料，我們發現 11 所在的節點分支度為 1 而且只有右子樹沒有左子樹，因此我們可以將 11 的父親節點 31 之左子樹鏈結指向 11 的右子樹樹根節點 20 即可，其結果如圖 6.24(b) 所示。

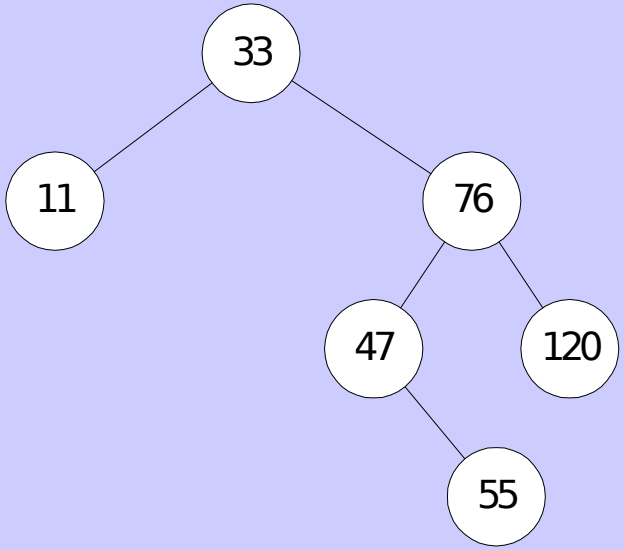
6.7.1 二元搜尋樹 (Binary Search Tree)

2. 若要刪除的資料所在的節點之左子樹不是空樹，我們必須將從該節點的右子樹中找出跟目前大於等於目前資料並且是最接近的節點（亦即尋找中序追蹤之後繼節點）來取代目前的節點。

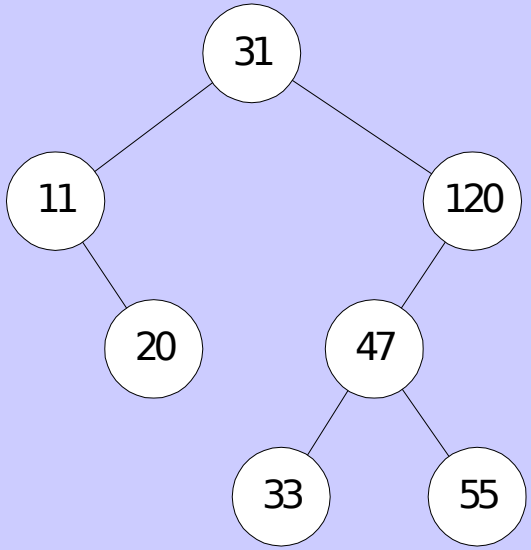


(a) 二元搜尋樹　　(b) 從圖 (a) 刪除 11 之結果

6.7.1 二元搜尋樹 (Binary Search Tree)



(c) 從圖 (a) 刪除 31 之結果



(d) 從圖 (a) 刪除 76 之結果

圖 6.24 二元搜尋樹

6.7.1 二元搜尋樹 (Binary Search Tree)

- 給定一棵二元搜尋樹，對該二元搜尋樹進行中序追蹤之結果相當於將所有的鍵值由小到大之排序。舉例來說，圖 6.24(a) 之二元搜尋樹經過中序追蹤結果為 11 、 20 、 31 、 33 、 47 、 55 、 78 、 120 。

6.7.2 高度平衡二元樹 (Height Balanced Binary Tree)

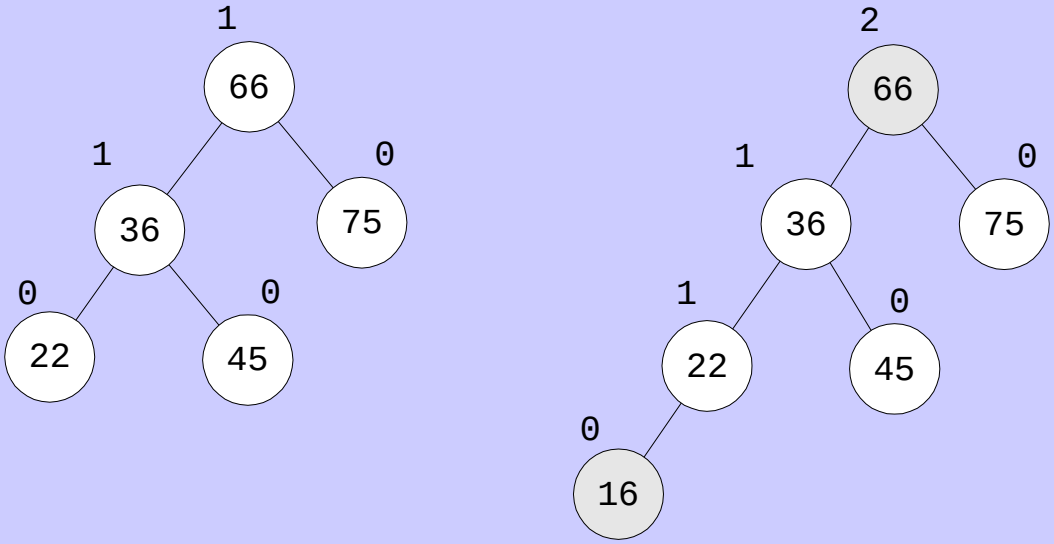
- 高度平衡二元樹（Height Balanced Binary Tree），又稱為 AVL 樹。目的是希望能夠建立一棵扁平矮胖的二元搜尋樹。

- 一棵 AVL 樹也是一棵二元搜尋樹，當資料插入二元搜尋樹或自二元搜尋樹刪除資料時均要檢查二元樹是否平衡，如非平衡狀態就須設法將之調整為平衡狀態。

- AVL 樹規定每一個節點上必須記錄該節點的平衡因子，在這裡平衡因子的定義為左子樹之高度減右子樹之高度。如果一棵二元搜尋樹的每一個節點之平衡因子皆屬於 -1、0 、或 1 三者之一時，我們將它稱為 AVL 樹。

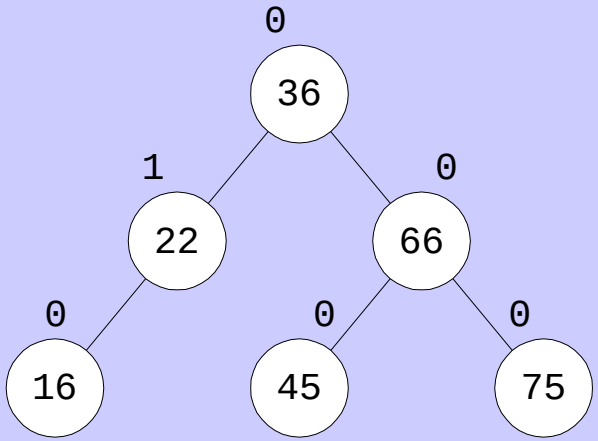
6.7.2 高度平衡二元樹 (Height Balanced Binary Tree)

- 根據插入資料所在的位置我們可以調整節點的處理方式分成四個類型： LL 型、 RR 型、 LR 型、 RL 型。



(a) 原本的 AVL 樹 (b) 加入 16 後的二元樹 (非 AVL 樹)

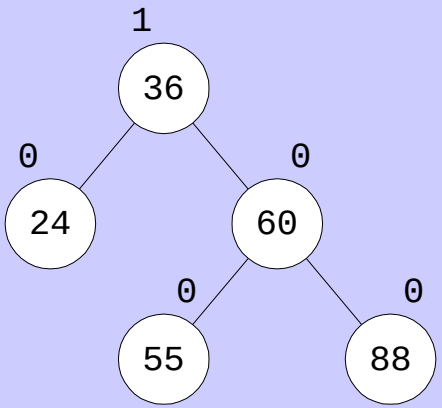
6.7.2 高度平衡二元樹 (Height Balanced Binary Tree)

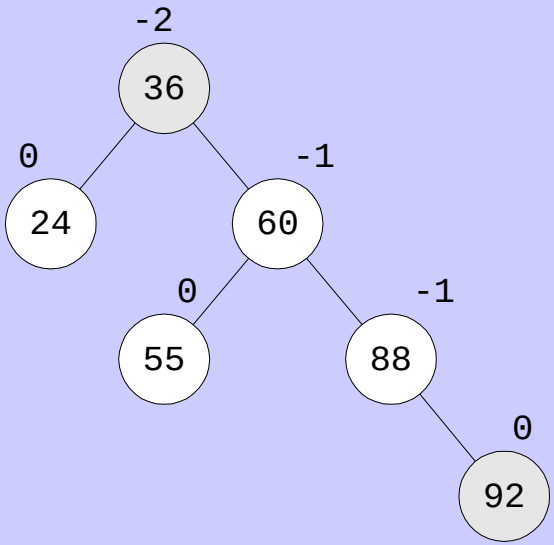


(c) 調整後之結果

圖 6.25 LL 型 AVL 樹調整處理

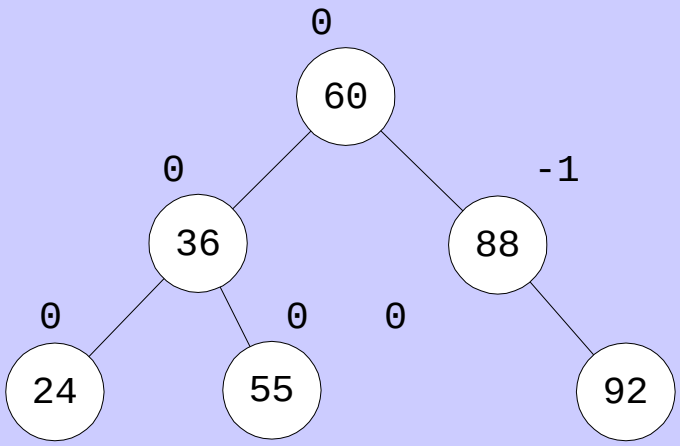
6.7.2 高度平衡二元樹 (Height Balanced Binary Tree)





(a) 原本的 AVL 樹　　(b) 加入 92 後的二元樹 (非 AVL)

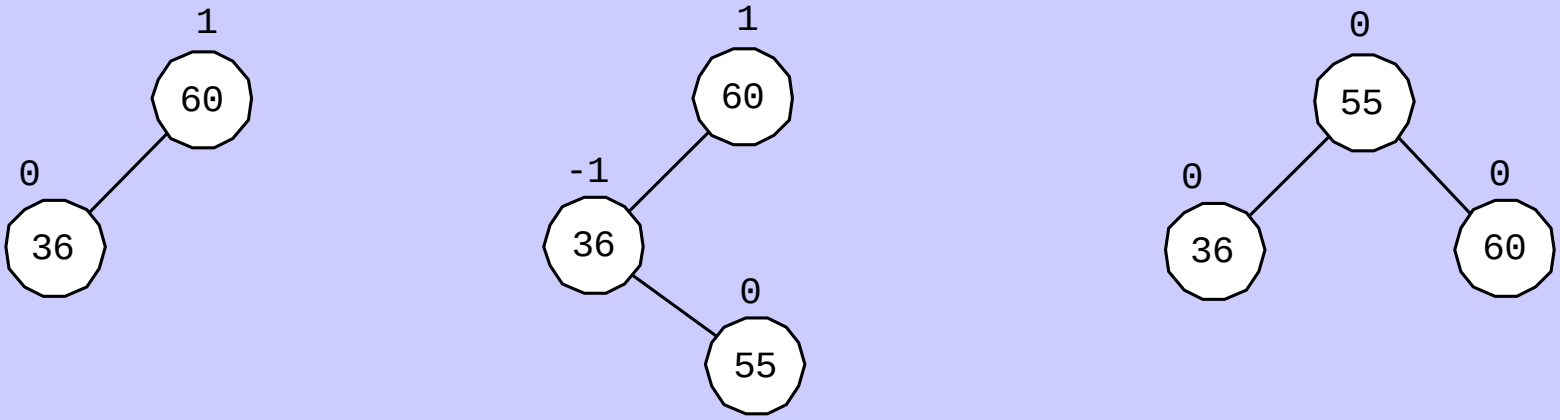
6.7.2 高度平衡二元樹 (Height Balanced Binary Tree)



(c) 調整後之結果

圖 6.26 RR 型 AVL 樹調整處理

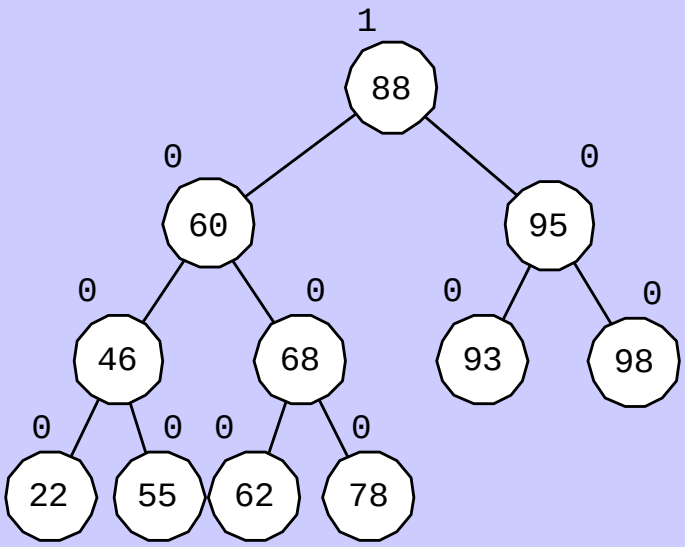
6.7.2 高度平衡二元樹 (Height Balanced Binary Tree)



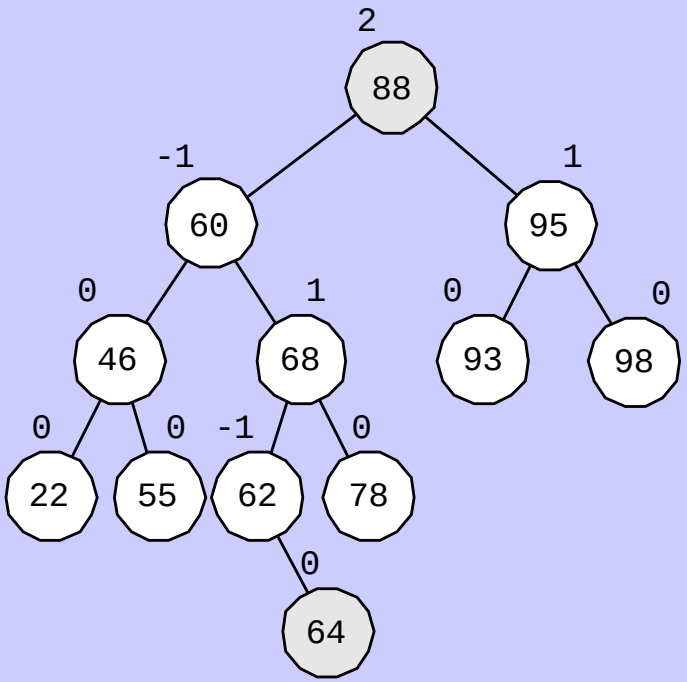
(a) 原本的 AVL 樹　　(b) 加入 55 後的二元樹　　(c) 調整後之結果

圖 6.27 第一種 LR 型 AVL 樹

6.7.2 高度平衡二元樹 (Height Balanced Binary Tree)

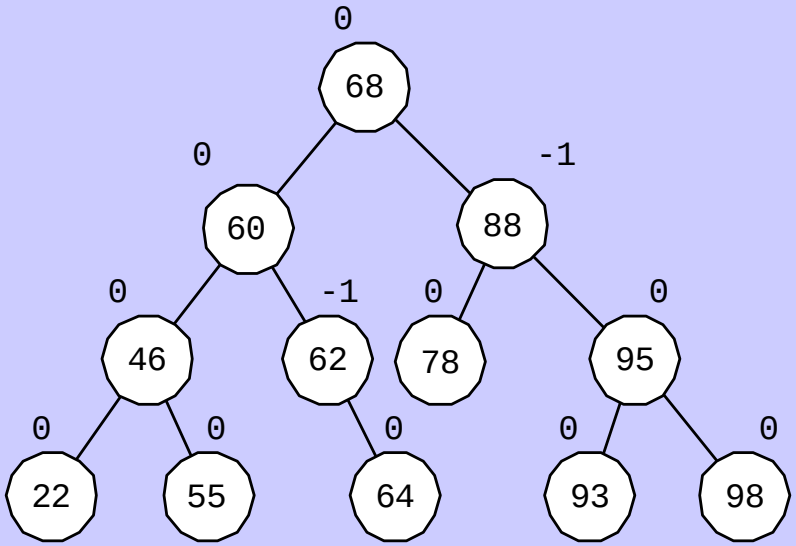


(a) 原本的 AVL 樹



(b) 加入 64 後的二元樹

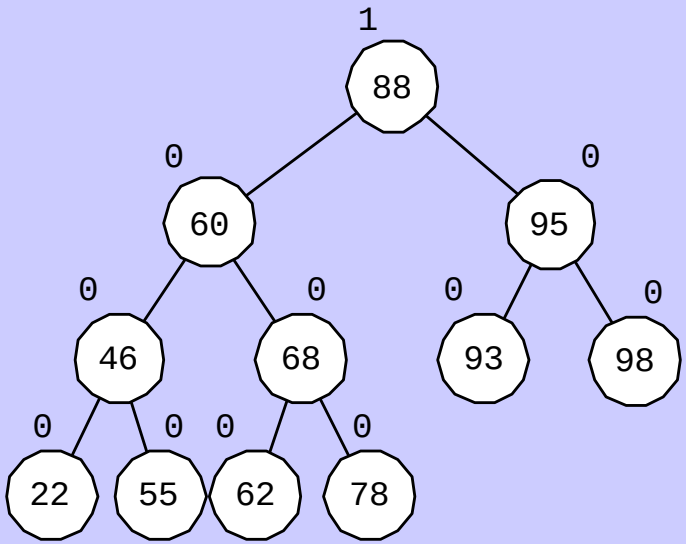
6.7.2 高度平衡二元樹 (Height Balanced Binary Tree)



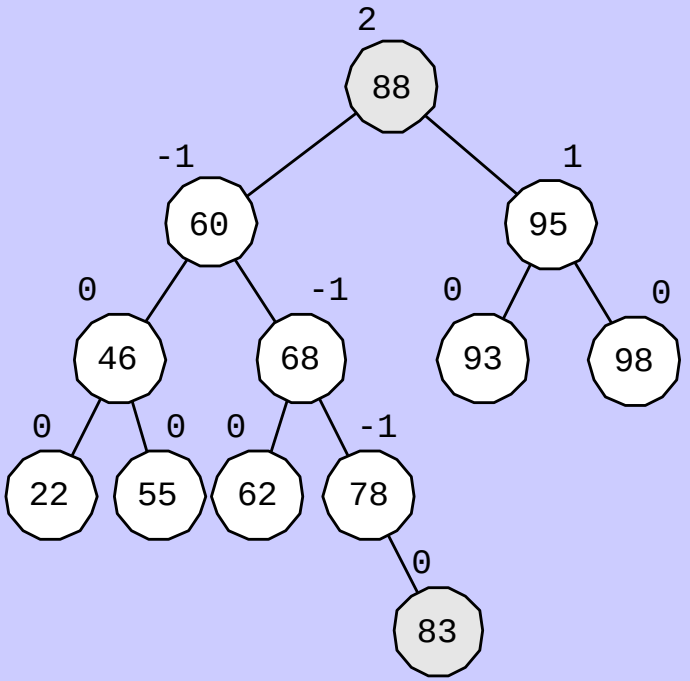
(c) 調整後之結果

圖 6.28 第二種 LR 型 AVL 樹

6.7.2 高度平衡二元樹 (Height Balanced Binary Tree)

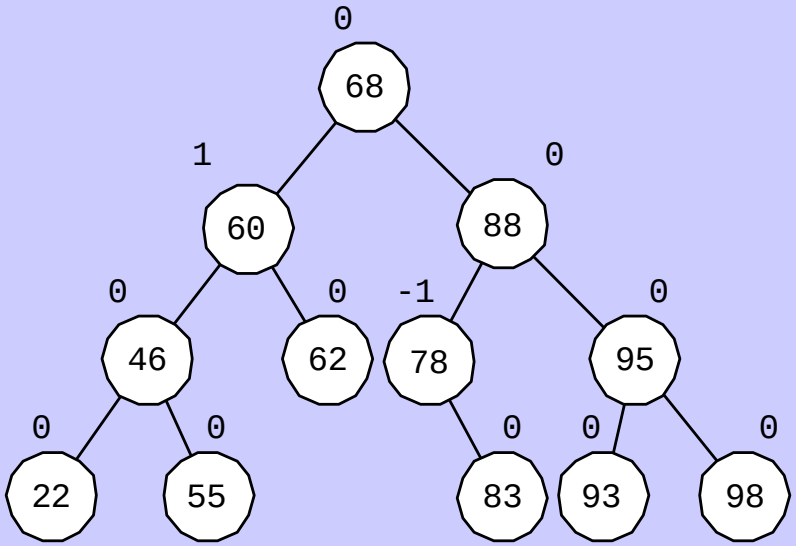


(a) 原本的 AVL 樹



(b) 加入 83 後的二元樹

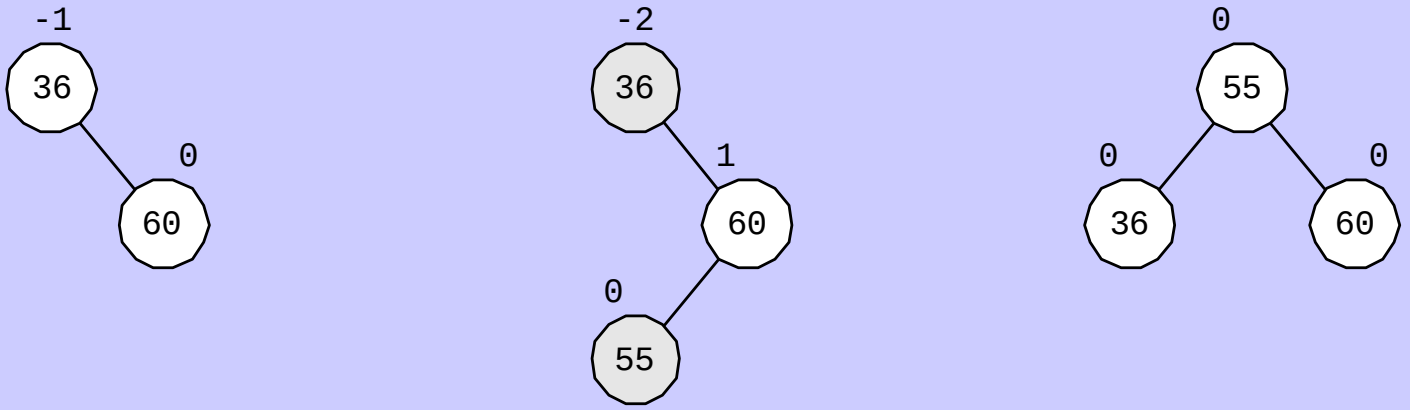
6.7.2 高度平衡二元樹 (Height Balanced Binary Tree)



(c) 調整後之結果

圖 6.29 第三種 LR 型 AVL 樹

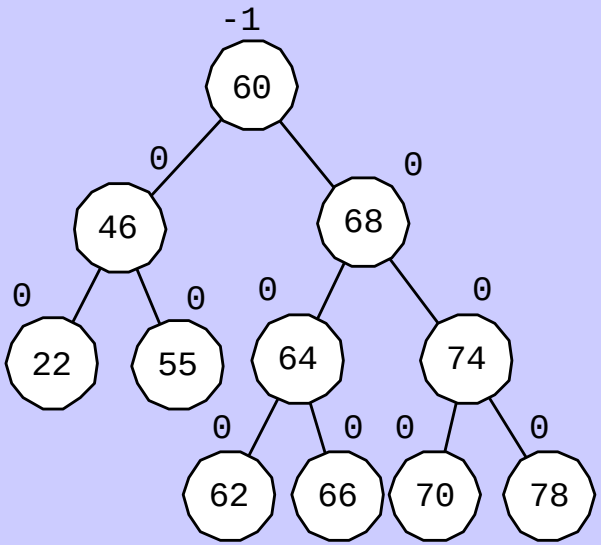
6.7.2 高度平衡二元樹 (Height Balanced Binary Tree)



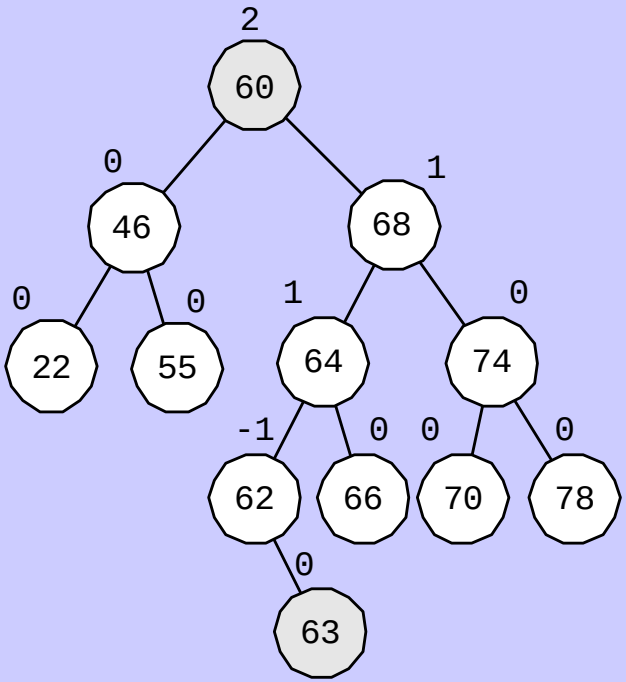
(a) 原本的 AVL 樹　　(b) 加入 55 後的二元樹　　(c) 調整後之結果

圖 6.30　第一種 RL 型 AVL 樹

6.7.2 高度平衡二元樹 (Height Balanced Binary Tree)

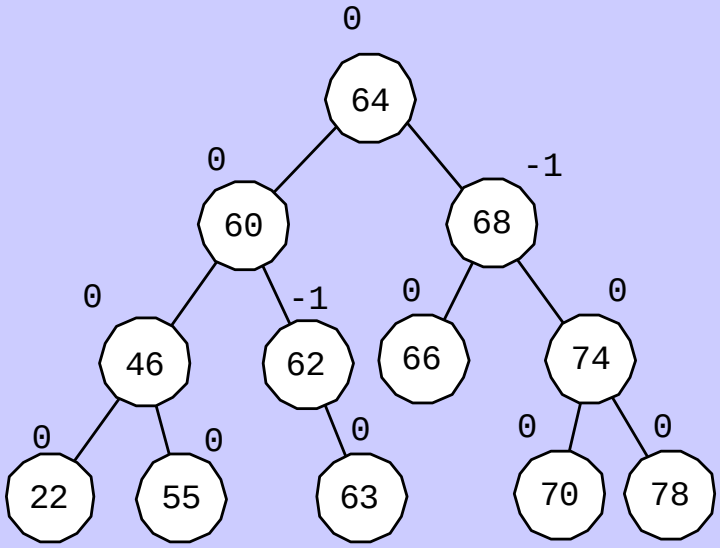


(a) 原本的 AVL 樹



(b) 加入 63 後的二元樹

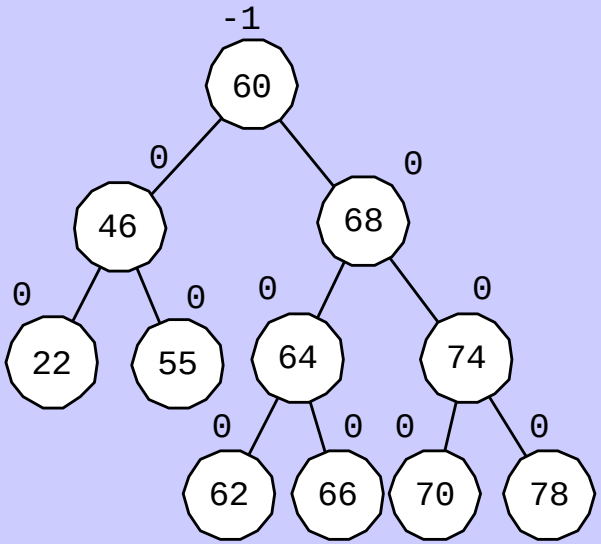
6.7.2 高度平衡二元樹 (Height Balanced Binary Tree)



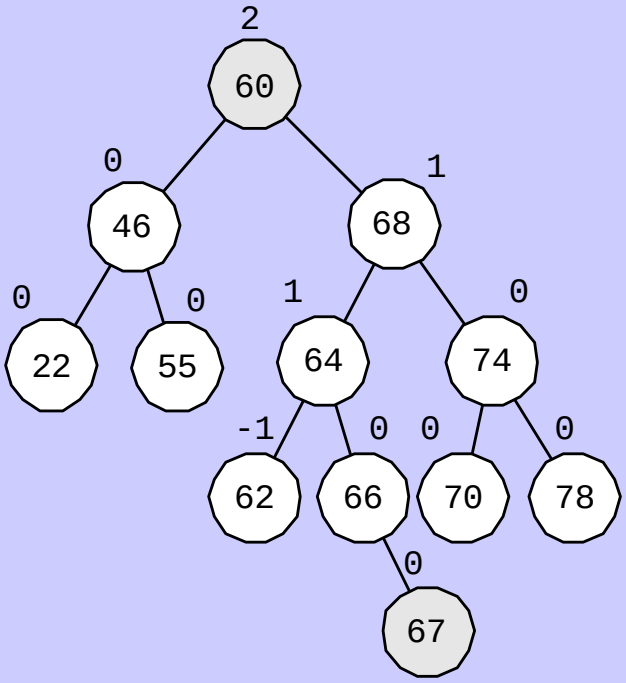
(c) 調整後之結果

圖 6.31 第二種 RL 型 AVL 樹

6.7.2 高度平衡二元樹 (Height Balanced Binary Tree)

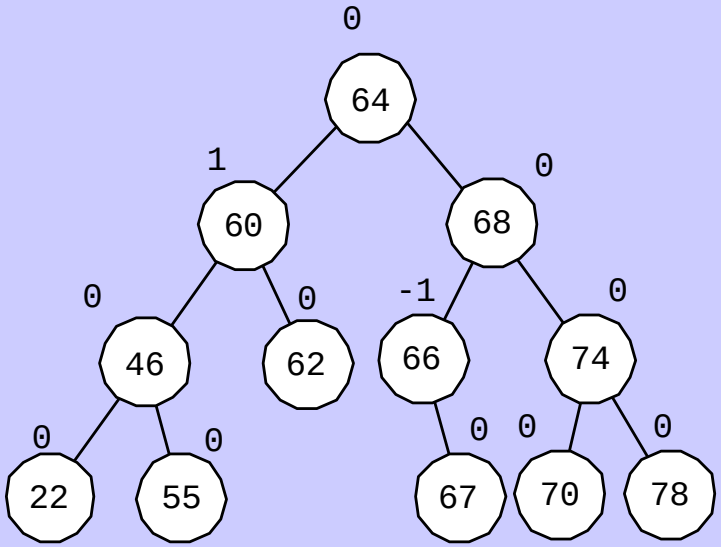


(a) 原本的 AVL 樹



(b) 加入 67 後的二元樹

6.7.2 高度平衡二元樹 (Height Balanced Binary Tree)



(c) 調整後之結果

圖 6.32　第三種 RL 型 AVL 樹

6.7.2 高度平衡二元樹 (Height Balanced Binary Tree)

- 二元搜尋樹較不利於資料之經常異動（插入、刪除），因為它不能保證在資料的異動之後仍然是保持扁平矮胖的長相。也就是二元搜尋樹比較適合儲存固定的資料，例如程式語言的保留字等。

- AVL 樹能夠在資料資料異動後仍然保持扁平矮胖的長相，因此適用於儲存經常異動的動態資料，例如編譯器 (Compiler) 裡的符號表 (Symbol Table) 等。

6.7.3 B 樹 (B Tree)

- 一個 m 階 B 樹結構就是一個高度 ≧ 1 的 m 階搜尋樹，樹上的每一節點均為 m 階節點，並滿足下列特性：

1. 樹根節點至少有兩個子節點，否則為空樹。
2. 個別節點之分支度均小於或等於 m 。
3. 個別節點最多含有 m-1 個鍵值。

- 4. 除了樹根節點外，所有非終端節點最少有⌈m/2⌉個子節點，最多有 m 個子節點。
- 5. 所有樹葉節點均在同一層，也就是說從樹根節點到任意樹葉節點所經之路徑長度均相同。
- 6. 高度增加的情形只發生於樹根節點一分為二時。

6.7.3 B 樹 (B Tree)

- 給定一棵 m 階 B 樹，其個別節點包含了 m-1 個鍵值資料 K_1、K_2、…、K_{m-1} 與 m 個鏈結資料 $P_{0,1}$、$P_{1,2}$、…、$P_{m-2,m}$、$P_{m-1,m}$。

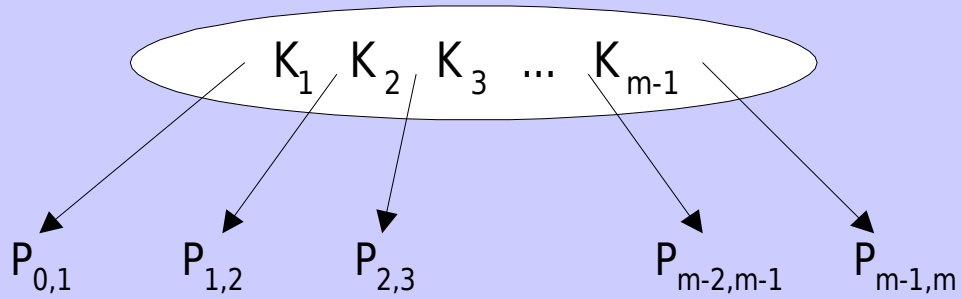


圖 6.34 m 階 B 樹的節點結構

6.7.3 B 樹 (B Tree)

- 圖 6.35 描述了一棵 4 階 B 樹結構的例子，由於每一個節點之分支度可能為 2 或 3 或 4 ，所以一棵 4 階 B 樹又可稱為 2-3-4 樹。

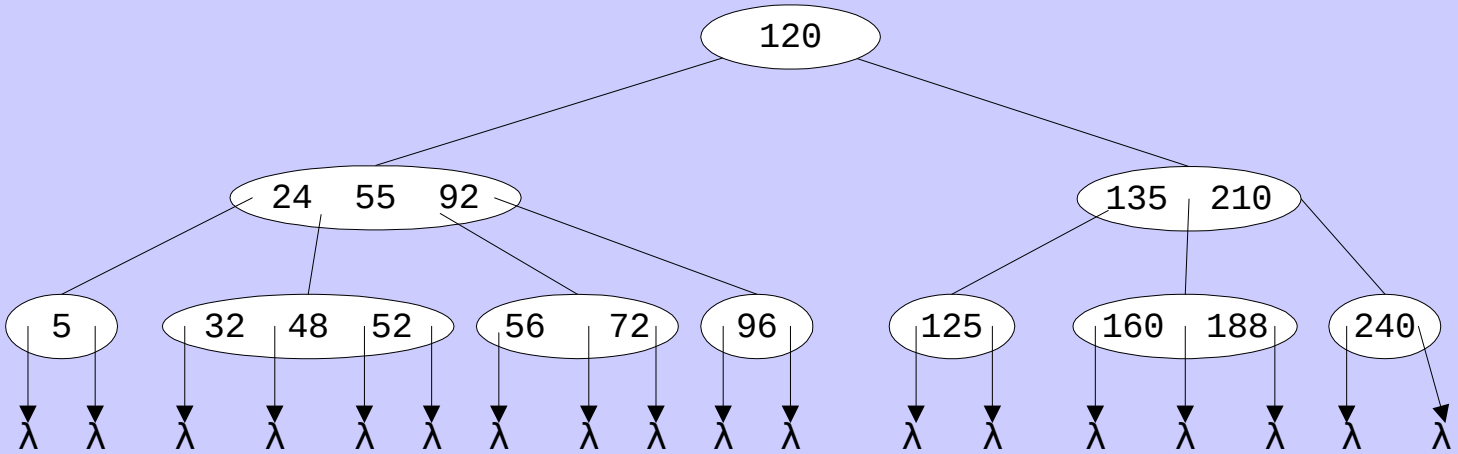
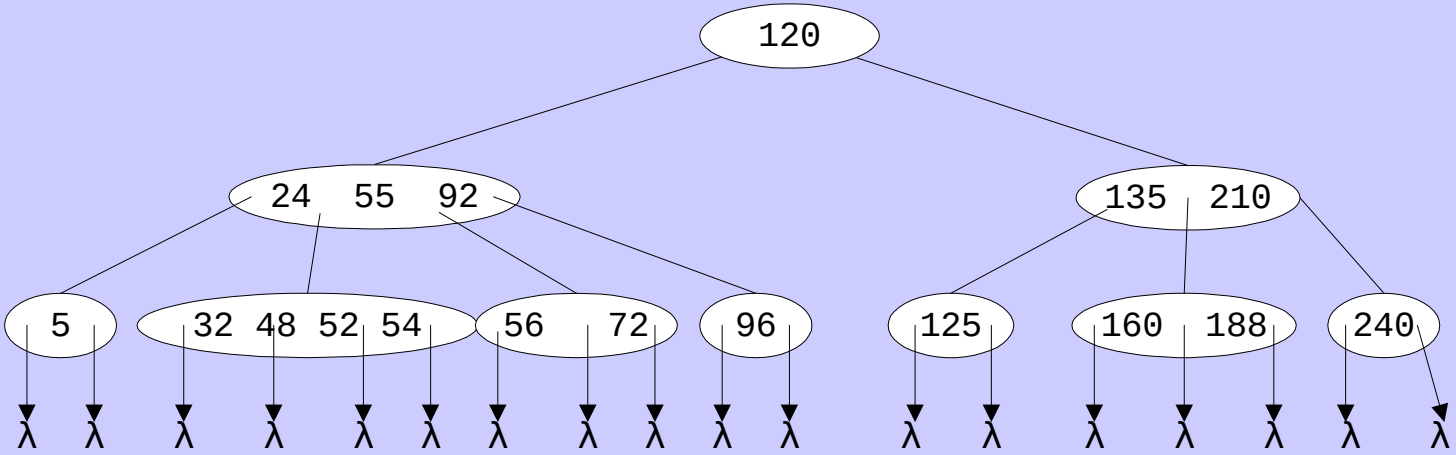


圖 6.35　4 階 B 樹

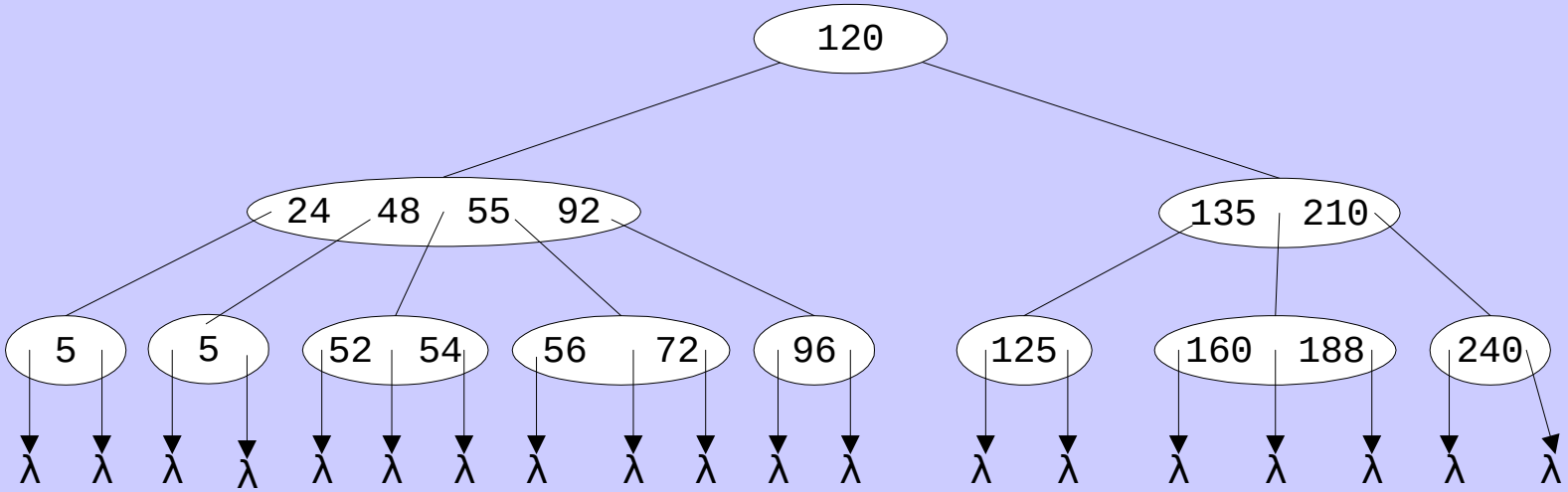
6.7.3 B 樹 (B Tree)

- 插入一個鍵值到給定一棵 m 階 B 樹，首先我們必須先找到插入位置的節點位置，若新建值資料的加入不會造成該節點的鍵值數爆滿，則直接將新資料置入節點中；否則須將該節點一分為二，以保持 m 階節點之特性。



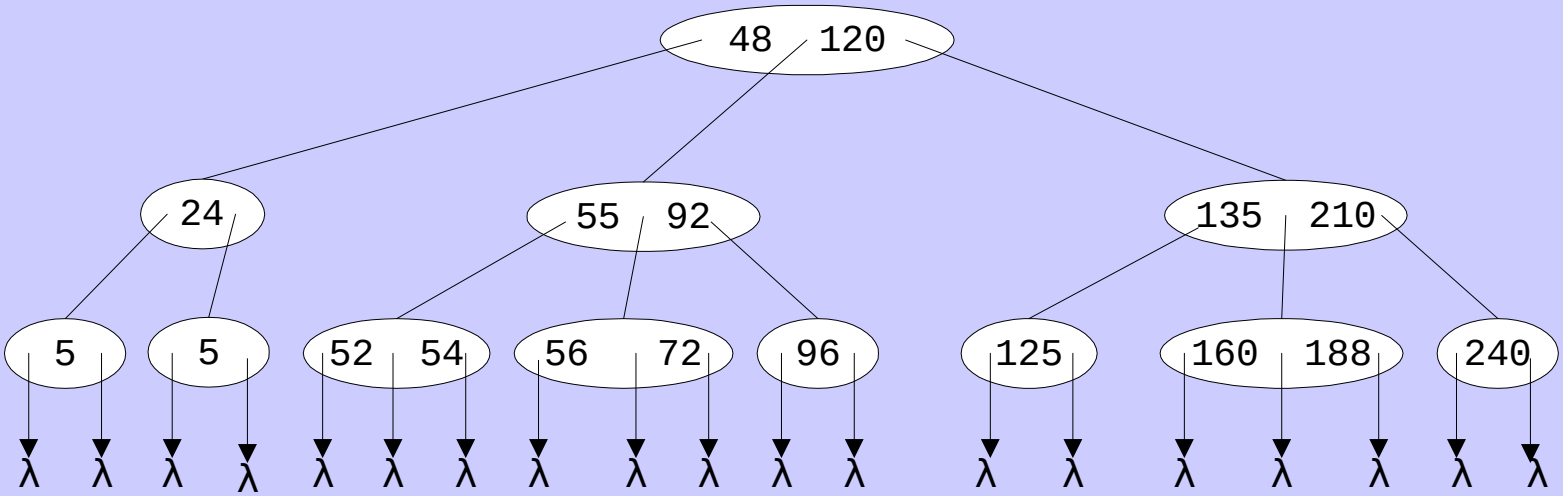
(a) 將 54 加入圖 6.35 之結果

6.7.3 B 樹 (B Tree)



(b) 圖 (a) 第一次節點分裂的結果

6.7.3 B 樹 (B Tree)



(c) 圖 (a) 第二次節點分裂的結果

圖 6.36　插入 54 之 4 階 B 樹處理過程

6.7.3 B 樹 (B Tree)

- 刪除一筆鍵值資料 K ，首先我們必須透過搜尋的處理找出該筆鍵值所在的節點位置，接著在將該筆資料刪除。由於鍵值資料所在的節點可能是樹葉節點或是非終端節點，其刪除的動作並不相同。

- 詳細的資料刪除程序如下所列：

步驟 (1)：令 NK = ⌈m/2⌉-1 ，且 NO(i) 表示節點 i 之鍵值個數。

步驟 (2)：找出欲刪除的鍵值 K 所在之節點 N 。

步驟 (3)：若 N 為樹葉節點，且刪除 K 之後， No(N) ≧NK ，則直接刪除之。

6.7.3 B 樹 (B Tree)

步驟 (4)：若 N 為樹葉節點，且刪除 K 之後， NO(N) < NK ，則分為以下三種情況處理：

a. 找出 N 節點的右邊兄弟節點 N^R ，如果 $NO(N^R) > NK$ ，則取出 N^R 節點中最小的鍵值 K^R 將之置於父親節點，並且將父親節點中第一個大於 K 之鍵值移到節點 N 。

b. 若右邊兄弟節點無法調整，則找左邊最靠近之兄弟節點 N^L ，如果 $NO(N^L) > NK$ 。那麼就將 N^L 中最大的鍵值移到父親節點，並將父親節點中第一個小於 K 之鍵值移到節點 N 。

6.7.3 B 樹 (B Tree)

c. 當左邊和右邊兄弟節點均無法調整時，那麼就應調整父親節點 N^P。在 N^P 節點中尋找鏈結指標 $P_{i+1,i+2}$ 其中 $K_i \leqq K < K_{i+1}$。順著 $P_{i+1,i+2}$ 找到 N^P 的子節點 N^C。若 $N^C = N$ 就沿鏈結指標 $P_{i,i+1}$ 找到 N^P 的子節點 N^C。將 N^P 中第一個大於 K 的鍵值 K_{i+1} 以及 N 節點中剩餘之鍵值全部移到節點 N^C。

步驟 (5)：處理非終端節點。令 $K_i = K$。沿著 $P_{i,i+1}$ 鏈結往下找到一個節點 NR，N^R 當中的第一個鍵值 K_1 是 B 樹中所有大於鍵值 K 中的最小者，然後將 K_1 移到 N 節點的 K_i 位置，若因而導致 No(NR) < Nk 則繼續處理節點 N^R（即跳回步驟 3）。

6.7.3 B 樹 (B Tree)

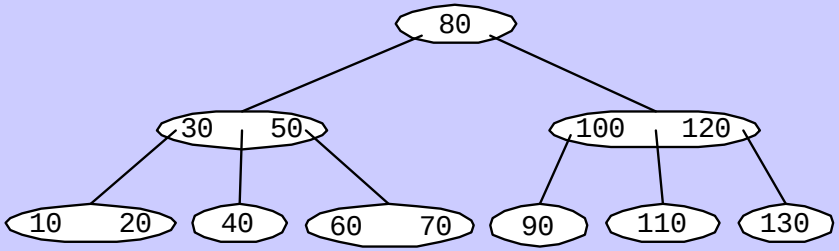


圖 6.37　原本的 3 階　B　樹

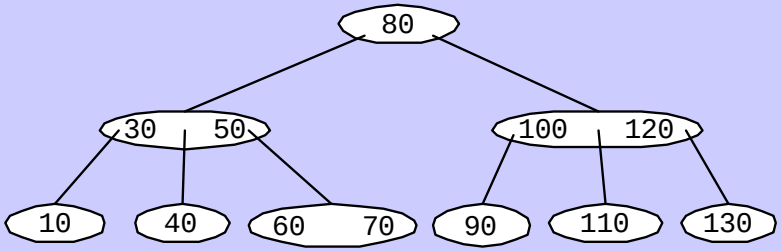


圖 6.38　從圖 6.37 中刪除 20 後之　B　樹

6.7.3 B 樹 (B Tree)

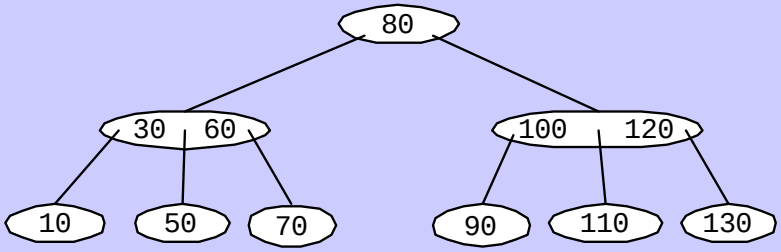


圖 6.39　從圖 6.38 中刪除 40 後之 B 樹

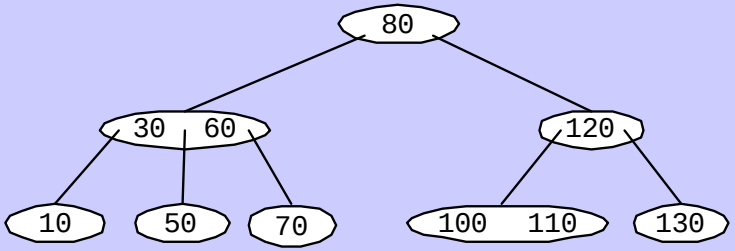


圖 6.40　從圖 6.39 刪除 90 後之 B 樹

6.7.3 B 樹 (B Tree)

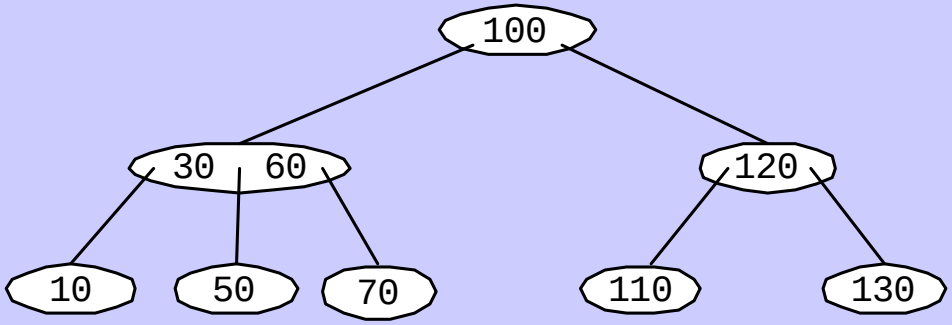


圖 6.41　從圖 6.40 刪除 80 後之 B 樹

6.8 樹狀結構的應用

- 使用二元樹來組織資料具有下優點：可以方便資料的插入和刪除，並且能夠提昇搜尋資料的速度，也可用來進行資料排序等等。本節將陸續介紹兩種常見的樹狀結構的應用。

6.8.1 集合 (Set) 的表示法和應用

- 樹林 (Forest) 是由兩棵或兩棵以上的樹所構成。
- 互斥集合 (Pairwise Disjoint Set) 就是兩兩沒有交集的集合
  - 。

【範例】圖 6.42 描述了一組互斥集合 S_1={A,B,C,D} ， S_2 ={E,F,G} ， S_3={H,I,J,K} ，我們發現任意兩個集合之交集皆為空集合。

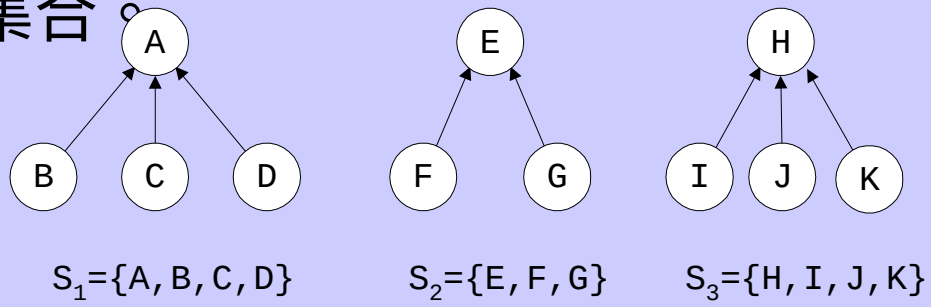


圖 6．42　用樹林來表示互斥集合

6.8.1 集合 (Set) 的表示法和應用

- 「聯集」是一個互斥集合常見的運算。將兩個分別以 X 與 Y 為樹根的集合進行聯集處理是將兩個集合的所有元素合併在一個集合中。我們通常利用 UNION(X,Y) 來表示以 X 與 Y 為樹根進行聯集的運算。

【範例】將圖 6.42 中兩個集合 S_1 與 S_2 進行聯集的處理。

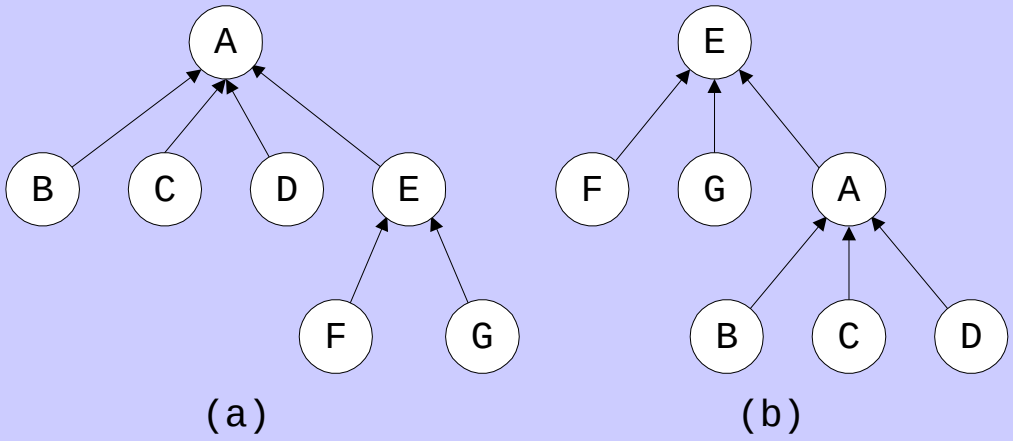


圖 6.43　S_1 U S_2 的兩種結果

6.8.2 霍夫曼樹與資料壓縮

- 霍夫曼編碼法（Huffman’s Encode）是一種無失真壓縮技術，其基本概念是將利用較短的編碼字來表示出現頻率高的字元；反之出現頻率較低的字元其編碼字的長度會較長。
- 建構一棵霍夫曼樹的處理過程列於下：

步驟 (1)：替每一個出現頻率不為零的字元產生一個樹葉節點，並將其出現頻率存於該樹葉節點的資料欄位。

步驟 (2)：令 L 是所有樹葉節點之集合。

步驟 (3)：當｜L｜樹葉節點的個數不等於 1 時，重複地執行下列子步驟：

a. 產生一個新節點 N。

6.8.2 霍夫曼樹與資料壓縮

b. 設定節點 N 為 N_1 和 N_2 的父親節點，N_1 和 N_2 是 L 中出現頻率最低的兩個節點。

c. 設定節點 N 的出現頻率等於 N_1 和 N_2 的出現頻率總和。

d. 從 L 中刪除 N_1 和 N_2，並將 N 加入 L 中。

步驟 (4)：標示樹中各個節點的右子樹鏈結為 0，左子樹鏈結為 1。

6.8.2 霍夫曼樹與資料壓縮

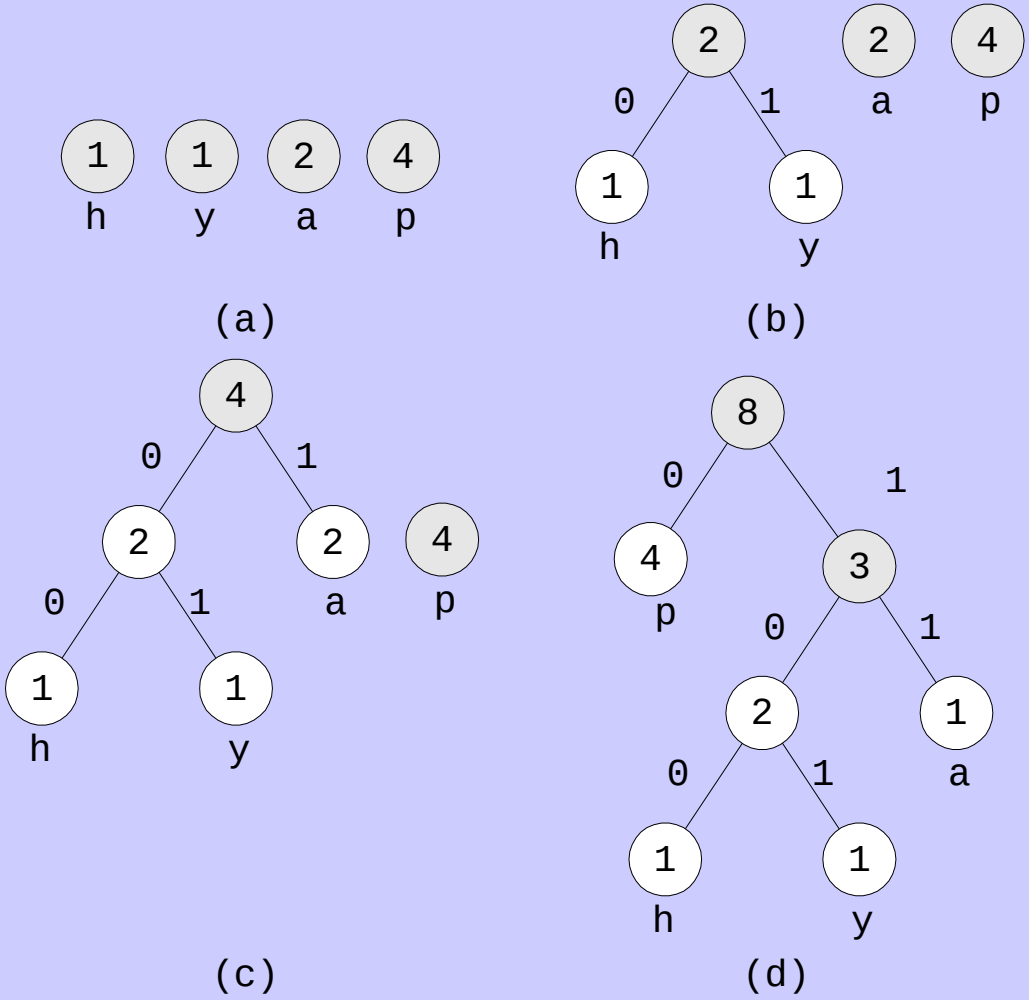


圖 6.45　"happy"　的霍夫曼樹建立過程

6.8.2 霍夫曼樹與資料壓縮

- 表 6.2 描述了字串 "happy" 的出現頻率與霍夫曼編碼的結果。

表 6.2 "happy" 的出現機率與霍夫曼碼

字元資料	出現頻率	霍夫曼碼	碼長度
h	0.125	100	3
a	0.25	11	2
p	0.5	0	1
y	0.125	101	3

6.8.2 霍夫曼樹與資料壓縮

- 給定一組已知出現頻率的資料，所建構出的霍夫曼樹是不唯一的。但是針對這組資料所需的平均編碼長度是固定的。